If, Then, Otherwise: Diagnosing Conditional Branching in Vision-Language Navigation

Anonymous CVPR submission

Paper ID *****

Abstract

Vision-language navigation agents are typically evaluated on their ability to follow route-like instructions toward a fixed goal. Real navigation instructions, however, often depend on the observed state of the environment: ***if*** *a condition holds,* ***then*** *follow one path,* ***otherwise*** *take another. Existing evaluations provide limited control over conditional branch execution, making it difficult to determine whether agents fail because of perception, grounding, navigation, or logical decision-making. We introduce* CondVLN*, a scene-graph-grounded benchmark and controlled testbed for conditional instruction following in vision-language navigation.* CondVLN *programmatically generates instructions whose branch conditions are grounded in verifiable 3D scene-graph predicates, with controlled variation in branch depth, dependency chain length, spatial composition, evidence observability, and instruction horizon. Our dataset holds 19,952 generated instructions across 34K objects and 1.88M spatial relationships in AI2-THOR, Matterport3D, Gibson, and ReplicaCAD, with 1,998 verified evaluation episodes, scored by standard VLN metrics and two branch diagnostics, Branch Selection Accuracy and Conditional Success Rate. Seven VLN agents (NaVid, NaVILA, CA-Nav, Open-Nav, VLN-Zero, StreamVLN, SNav-7B) reach a named object in a few percent of episodes, and most of their successes on branching episodes bypass the required branch, a failure that success rate and path length do not show. A neurosymbolic agent that checks each condition explicitly follows the correct branch on significantly more episodes, showing what condition following buys. Ablations on identical episodes show that the remaining potential performance gain is in acquiring the necessary evidence, not in choosing the correct branch. Project page:* https://condvln.github.io.

1. Introduction

Vision-language navigation (VLN) has emerged as a central testbed for evaluating reasoning capabilities of embodied AI, requiring agents to ground natural-language instructions into sequential decisions in 3D environments. Existing benchmarks such as R2R [1], RxR [14], AI2-THOR [12], Gibson [23], ReplicaCAD [19], and Habitat-based extensions [19] have significantly advanced multimodal semantic and spatial perception. However, the dominant focus of these benchmarks remains goal-directed navigation, where evaluation focuses on whether agents can reach target locations following instruction trajectories, with limited emphasis on conditional reasoning, branching behavior, and temporal dependency tracking.

In real-world embodied settings, navigation agents must frequently make decisions conditioned on environment state, such as: *If there's a coffee table in the kitchen, head to the stool north of the dishwasher in the kitchen, otherwise make your way to the light below the sink in the bathroom.* These instructions require agents to jointly perform semantic and spatial grounding while also reasoning over explicit conditional structures that govern action selection. Existing datasets neither explicitly control these conditions nor systematically use them for evaluation, limiting controllability over reasoning complexity.

To address this gap, we introduce CondVLN, a benchmark for conditional reasoning in vision–language navigation that makes conditional reasoning an explicit and controllable evaluation axis. CondVLN is a controlled and systematic testbed for conditional instruction following: the truth of every condition is known by construction, every episode is verified to be solvable, and the branch taken is measured directly. Starting from hierarchical 3D scene graphs that encode visually grounded semantic-spatial relations extracted from embodied environments, we programmatically generate navigation tasks with fine-grained control over logical complexity, including branch depth, dependency chain length, cross-room composition, and instruction horizon. A key property of CondVLN is that all con-

ditional logic is grounded in explicit scene-graph predicates over object–object and object-room spatial relations, ensuring that branching decisions are semantically verifiable and unambiguous. This design separates branch following from goal reaching for any agent.

CondVLN is fully compatible with existing VLN ecosystems (e.g., VLN-CE [13]), enabling evaluation across existing simulators and models without significant infrastructure changes. Beyond standard success metrics, we introduce structured reasoning-specific metrics and evaluation axes for diagnosing conditional navigation failures. By varying instruction length, logical depth, conditional branching structure, and scene diversity, CondVLN enables fine-grained diagnostic analysis of agent behavior.

Empirical evaluation with state-of-the-art VLN models, including zero-shot instruction-following agents, shows that they do not handle conditional instructions: they rarely reach the named target, and most of their successes bypass the required branch.

Our main contributions are as follows:

1. We introduce conditional reasoning as an explicit evaluation axis for VLN in addition to semantic and spatial reasoning, enabling a holistic and systematic study of neurosymbolic reasoning in embodied navigation.
2. We propose a hierarchical-scene-graph-based task generation framework that produces semantically verifiable conditional navigation tasks with controllable complexity over branch depth, dependency chains, and spatial composition.
3. We construct a cross-environment VLN benchmark spanning MP3D, Gibson, AI2-THOR, and ReplicaCAD, enabling evaluation across diverse embodied settings without significant infrastructure changes.
4. We introduce controlled reasoning-specific evaluation metrics that disentangle failures due to conditional reasoning through structured variation of task properties.
5. We demonstrate that state-of-the-art VLN models perform poorly on conditional instructions, establishing CondVLN as a benchmark for neurosymbolic reasoning in embodied AI.
6. We provide a neurosymbolic agent that checks each condition explicitly to demonstrate the effect of explicit condition checking. Ablations on identical episodes attribute the potential additional performance gain to perception and exploration rather than reasoning.

2. Related Works

Recent advancements in Large Vision-Language Models (VLMs), such as NaVid [25], NavGPT-2 [26], NaVILA [5], and OmniVLA [11], utilize various methods to achieve better performance on VLN for standard benchmarks such as R2R-CE and RxR-CE [1, 14]. Zero-shot navigation via foundation models has also been explored by methods such as Open-Nav [16] and CA-Nav (Constraint-Aware Navigator) [4] as an alternative to expert demonstrations, and has achieved good performance in unconventional environments on easily comprehensible instructions as well. VL-Nav [9] adds a VLM task planner over an online scene graph to heuristic frontier exploration, but its subtasks still form a linear list with no branches and no absence conditions. The benchmarks these agents are evaluated on share that limitation: they either focus primarily on linear instruction following rather than complex conditional logic, or they only use conditional instructions without any organized and structurally verifiable way of assessing them. Thus, these evaluations say little about how agents handle long-horizon tasks in which the conditional logic is the task itself rather than an add-on to route following.

In particular, other benchmarks like VLNVerse [15], and IEDL [20] using datasets including AI2-THOR [12], Gibson [23], ReplicaCAD [19], R2R [1], and RxR [14] primarily focus on whether agents can reach a singular identifiable goal, and so the style of tasks tested by these benchmarks is not extensible to longer-term tasks with logical complexity. Though there has been some progress in testing navigation over long-horizon, multi-part tasks, such as with CoNavBench [21], HA-VLN 2.0 [8], and SeqWalker [10], there are few benchmarks or frameworks that appropriately address conditional reasoning, branching behavior, or temporal dependency tracking. These are all tasks characteristic of long-horizon real-world navigation goals and which CondVLN is designed around. Conditional instructions have begun to appear in recent benchmarks, each covering one facet of the problem. The Environment State category of NavSpace [24] appends a single condition to the end of an R2R route ("if you see the keys, stop, otherwise go to the front door"); the object is visible from the final viewpoint and success is the endpoint within 2 m, so the setting tests whether an agent reads the condition at the goal. LH-VLN [17] chains sequential subtasks, and its conditional success rate measures the dependency between stages. VLN-NF [18] tests false-premise instructions whose target is absent, the absence case without an alternative branch. CondVLN brings these facets together: every condition is a scene-graph predicate whose truth is known by construction; conditions decide the destination and are composed through controlled nesting depth and chain length; true and false conditions each lead to their own branch, so both outcomes of a decision are tested; and branch selection accuracy and conditional success rate verify the branch the agent takes. Together these make CondVLN a controlled and systematic testbed for conditional instruction following.

3. Benchmark Design

CondVLN features a programmatic benchmark generation framework for evaluating neurosymbolic reasoning in VLN,

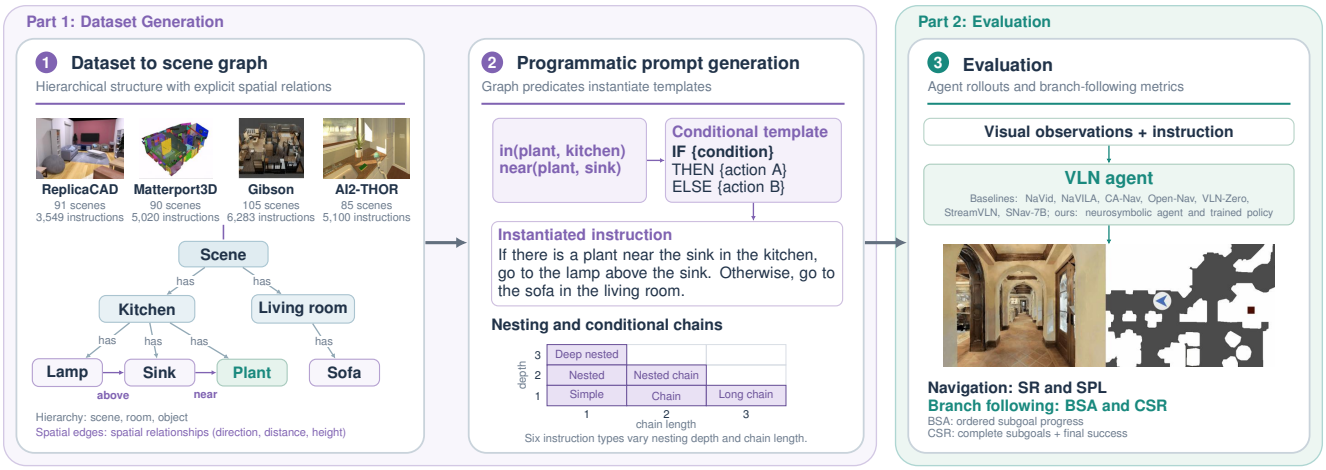


Figure 1. Overview of CondVLN, in two parts. Dataset generation: 1 scenes from four environment sources become hierarchical scene graphs with explicit spatial relations (statistics in Table 2); 2 graph predicates instantiate conditional templates, six instruction types vary nesting depth and chain length, and the truth of every condition is known by construction. Evaluation: 3 agents receive the camera observations and the instruction text, not the scene graph, condition truths, or goal location; metrics are success rate (SR), success weighted by path length (SPL), branch selection accuracy (BSA), and conditional success rate (CSR).

where agents must combine semantic-spatial grounding in embodied environments with symbolic reasoning over explicit logical and conditional structures. Figure 1 shows the two-part framework: dataset generation and evaluation.

3.1. Dataset Generation

Dataset generation starts from dataset-specific scene hierarchy files (Sections 3.1.1 and 3.1.2) and converts them into symbolic scene graphs with semantic and spatial relationships (Section 3.1.3). These graphs are then used to synthesize conditional navigation instructions with controllable logical complexity (Section 3.1.4), which are finally converted into VLN-CE-compatible navigation episodes for evaluation in Habitat-based environments (Section 3.1.5).

3.1.1. Source Datasets and Scene Representations

CondVLN is instantiated across multiple indoor 3D environment sources to evaluate whether conditional navigation generalizes across both synthetic and scan-based scenes. We include synthetic or simulator-native environments, such as AI2-THOR [6, 7, 12] and ReplicaCAD [19], as well as scan-based environments, such as MP3D [3] and Gibson [23]. These sources differ in scene scale, annotation format, object taxonomy, and geometry representation.

Datasets built on synthetic data are generated procedurally within a simulator or modeling framework, which can reduce annotation noise and provide complete ground-truth metadata. For ReplicaCAD we compose each scene GLB from its configuration together with the stage and object GLB assets. For AI2-THOR we export each scene as one GLB with its materials baked into textures. Navigation meshes are generated for both.

We also utilize another category of datasets generated based on scans of real-world physical scenes invoking techniques such as RGB-D scanning or photogrammetry. This includes the MP3D dataset, a database of 90 buildings captured with an RGB-D camera rig. The object inventories of the two scan-based sources are automatic 3D scene graphs built on the scans (categories with 3D boxes).The single-room sources use the object lists from the simulators.

3.1.2. Scene Hierarchy Input

As each environment source stores semantic and geometric scene information differently, we first convert dataset-specific annotations into a common hierarchy representation. Raw labels are mapped to canonical object categories by a per-source glossary (Section S6). We abstract these heterogeneous sources into a unified scene hierarchy that is represented as a collection of rooms, and each room contains object instances with semantic labels and geometric metadata. At minimum, each object is associated with a 3D center location. When available, the hierarchy may also include object size, radius, floor area, or an explicit 3D axis-aligned bounding box (AABB), which are generated based on the given annotations extracted from the datasets. Formally, for each object o_i, we denote its semantic label by ℓ_i and its 3D center by $\mathbf{c}_i = (x_i, y_i, z_i)$. The coordinate convention follows the simulator frame, where the XZ plane is treated as the horizontal ground plane and the Y axis is vertical. If an object has a size vector $\mathbf{s}_i = (s_i^x, s_i^y, s_i^z)$, we derive AABB from the object center and extent. If an explicit AABB is provided, it is used directly. Optional radii are used as a fallback geometric approximation when bounding boxes are unavailable. The hierarchy parser produces a room-level symbolic representation containing object nodes, room containment edges, and available geometric metadata. This representation serves as the foundation for extracting object-object spatial relations and generating

scene-graph predicates for conditional instruction synthesis.

3.1.3. Semantic and Spatial Relationship Generation

The spatial relationship generator converts each room-level object set into a directed relation graph. Each object is represented as a node, and edges encode both canonical geometric relations and thresholded semantic relations. These relations serve as the symbolic link between spatial-semantic scene structure and the logical conditions used for instruction generation.

Distance metric selection. For each ordered object pair (o_i, o_j) in the same room, we compute the relative displacement from source object o_i to target object o_j as $\boldsymbol{\delta}_{i\to j} = \mathbf{c}_j - \mathbf{c}_i = (\delta_x, \delta_y, \delta_z)$, with the center distance $d_{ij}^{\text{center}} = \|\boldsymbol{\delta}_{i\to j}\|_2$, and the horizontal ground-plane distance $h_{ij} = \sqrt{\delta_x^2 + \delta_z^2}$.

Since source datasets vary in geometric detail, the CONDVLN selects the distance used for semantic thresholding with a prioritized fallback: d_{ij}^{used} is selected in the order $d_{ij}^{\text{AABB}} \to d_{ij}^{\text{sphere}} \to d_{ij}^{\text{center}}$. Here, $d_{ij}^{\text{AABB}} = \sqrt{\Delta_x^2 + \Delta_y^2 + \Delta_z^2}$ is the surface-to-surface AABB distance, where $\Delta_a = \max(a_i^{\min} - a_j^{\max}, a_j^{\min} - a_i^{\max}, 0)$ for each axis $a \in \{x, y, z\}$, and $d_{ij}^{\text{sphere}} = \max(0, d_{ij}^{\text{center}} - (r_i + r_j))$ is the bounding-sphere surface distance. This fallback allows the same generation framework to operate across datasets with heterogeneous metadata. We prioritize AABBs when available because they capture object extent and face-level geometry with less error.

Canonical directional predicates. To generate canonical 3D directional relations, we compute the azimuth and elevation of the displacement vector. Azimuth is discretized into eight equal compass sectors: {`east`, `north-east`, `north`, `north-west`, `west`, `south-west`, `south`, and `south-east`}. Elevation is discretized into {`up`, `level`, and `down`}. The final canonical predicate composes the azimuth and elevation bins. Level relations are expressed as horizontal predicates such as `east of`, while non-level relations include both horizontal and vertical components, such as `east-and-up of`. These canonical predicates provide a compact symbolic description of relative 3D layout.

Semantic relation predicates. In addition to canonical geometric relations, we generate semantic predicates that are closer to real-world natural-language navigation instructions. These predicates are emitted using conservative metric thresholds: `near` and `far from` are based on d_{ij}^{used}, `higher than` and `lower than` are based on vertical displacement δ_y, and `above` and `below` require both sufficient vertical displacement and small horizontal separation. We use fixed thresholds (near ≤ 1.0 m, far ≥ 4.0 m, higher or lower $|\delta_y| \geq 0.4$ m, above or below $|\delta_y| \geq 0.4$ m with horizontal separation ≤ 2.0 m) for reproducibility across generated datasets.

Scene graph construction. For each room, we create a directed graph containing room-object containment edges and object-object relation edges. Each object is assigned a unique identifier and stores its semantic label, center location, and any available geometric metadata. Every ordered object pair includes a canonical directional edge and any semantic edges whose thresholds are satisfied. Each relation edge stores the subject object, predicate, target object, predicate type, and the metrics used to generate the predicate, including center distance, sphere distance, AABB distance, selected distance type, azimuth bin, and elevation bin. This makes each symbolic relation in the final scene graph traceable to the underlying metric computation.

3.1.4. Conditional Instruction Synthesis

Building on the constructed scene graphs, we synthesize conditional navigation instructions that require agents to reason over object presence and spatial context, and ultimately combine spatial-semantic grounding with symbolic branch execution. Each instruction is generated by grounding the previously generated symbolic predicates from the scene graph into natural-language conditions and navigation actions. They form logical structures such as *conditional branches*, *nested decisions*, and *multi-step chains*. As every condition and target is grounded in real object instances and relationships from the scene graph, the generated instructions remain consistent with the underlying 3D environment and the truth value of each condition is known at generation time.

Object sampling strategy. We construct conditional instructions using a structured object sampling process over the scene graph. Objects are sampled to serve two roles: *valid* references, corresponding to objects that exist in the room, and *invalid* condition references, corresponding to objects that are absent from a given room. Invalid conditions name a category from the environment-wide vocabulary that is absent from the named room under canonical labels and hypernyms; in single-room scenes it is absent from the whole scene, and in multi-room houses it may exist in another room, which tests room grounding. To ensure clarity, objects with identical semantic labels within a room (e.g., multiple "lamps" in the room) are disambiguated using spatial qualifiers derived from the scene graph (e.g., "lamp near the bed"), and objects that cannot be uniquely identified are excluded. This guarantees that all sampled objects are uniquely referable and grounded without ambiguity.

Table 1. Taxonomy of instruction complexity: depth is the nesting level, chain length the number of sequential branches; the verified episodes are balanced across the six categories.

Category	Depth (nesting)	Chain Length (branches)	Description
Simple	1	1	Single IF-ELSE
Nested	2	1	IF within IF
Deep Nested	3	1	Multi-level nesting
Chain	1	2	IF / ELSE IF
Long Chain	1	3	Multi-branch chain
Nested Chain	2	2	Nested + chained

Instruction templates and complexity. Conditional instructions are generated from structured templates that map scene graph predicates into logical forms. Each instruction follows a conditional structure of the form

```
IF condition THEN action ELSE action
```

where `condition` corresponds to an object presence query and `action` corresponds to navigation targets. This base form can be extended either by nesting conditionals (e.g., `IF A THEN (IF B THEN x ELSE y) ELSE z`) or by chaining multiple branches (e.g., `IF A THEN x ELSE IF B THEN y ELSE z`). To systematically control reasoning difficulty, we define a set of instruction categories that vary along two axes: *logical depth*, corresponding to the level of nesting, and *chain length*, corresponding to the number of sequential conditional branches. As shown in Table 1, these categories allow us to evaluate navigation agents under progressively more complex decision-making scenarios; they range from single-step condition evaluation to multi-step branching and hierarchical reasoning.

3.1.5. VLN-CE Dataset Realization

To make CondVLN directly usable with existing VLN evaluation frameworks, we convert generated conditional instructions into Habitat-compatible VLN episodes. For each task, we extract start and goal coordinates associated with the selected branch targets from the scene graph and transform coordinates from dataset-native conventions into the Habitat coordinate frame. A Habitat-Sim pathfinder is then used to compute geodesic shortest paths between start and goal states. Episodes with failed pathfinding are filtered out, and the remaining trajectories are stored as reference paths together with geodesic distances, enabling standard VLN metrics. Each output episode follows the R2R/VLN-CE JSON format, including the scene identifier, start and goal positions, rotation, instruction text, reference path, and geodesic metadata. This step preserves the symbolic conditional structure of each generated instruction while packaging it as a standard navigation episode. As a result, CondVLN can be evaluated using existing frameworks (e.g., VLN-CE, VLN-Zero, NaVid) by replacing the dataset paths in the corresponding configuration files without significant simulator modifications. Figure 1 and Table 2 summarize the resulting benchmark coverage. The verified episodes in Table 2 are the evaluation sets of Section 5: audited based on checks in Section S6 (canonical labels, explicit coordinate frames, navmesh-snapped start, subgoals, and goal) and kept only if a privileged shortest-path controller completes them within the step budget.

Table 2. Benchmark statistics. Totals are sums, except categories, counted as distinct; spatial relations are object-to-object edges (direction, distance, height) plus a room-membership edge per object.

	ReplicaCAD	AI2-THOR	Gibson	MP3D	Total
Scenes	91	85	105	90	371
Objects	2,293	4,003	7,487	20,279	34,062
Object categories	41	127	55	146	274
Spatial relations	255,529	560,219	146,685	918,059	1,880,492
Generated instructions	3,549	5,100	6,283	5,020	19,952
Verified episodes	500	500	499	499	1,998
Mean reference path (m)	10.0	9.9	22.5	33.5	19.0
Episodes with subgoals	66%	69%	67%	66%	67%

3.2. Evaluation Metrics

CondVLN is designed not only as a dataset, but also as a controlled evaluation suite for diagnosing reasoning failures in embodied navigation. Standard VLN metrics, such as success rate (SR), oracle success rate (OSR), distance to goal, and success weighted by path length (SPL), measure whether an agent reaches a target efficiently. However, they do not distinguish failures in visual navigation from failures in conditional reasoning or symbolic branch selection. Thus we introduce reasoning-specific metrics for conditional prompts: branch selection accuracy (BSA) and conditional success rate (CSR). BSA captures branch-consistent progress without requiring final task success, while CSR further requires both complete branch following and standard Habitat success.

Branch selection accuracy. Branch Selection Accuracy (BSA) measures how much of the correct conditional branch the agent follows. Let $G_i = (g_{i,1}, \ldots, g_{i,m_i})$ denote the ordered subgoals associated with that branch for episode i, and let k_i be the number of leading subgoals of G_i the agent reaches in order within the Habitat success radius τ, counting up to the first miss. For episodes with $m_i > 0$, we define $\mathrm{BSA}_i = k_i/m_i$, allowing fractional credit for partial branch progress. For episodes with no annotated subgoals (every condition false; one third of the verified episodes), branch progress is not separately observable from reaching the goal, so $\mathrm{BSA}_i = 1$ if the agent is ever within τ of the goal and 0 otherwise. No episode is excluded: $\mathrm{BSA} = N^{-1} \sum_{i=1}^{N} \mathrm{BSA}_i$ over the same N episodes as SR and SPL.

Conditional success rate. Conditional Success Rate (CSR) measures strict conditional task completion by requiring both complete branch following and final navigation success. We define $\mathrm{CSR}_i = \mathbb{1}[\mathrm{BSA}_i = 1 \wedge \mathrm{Success}_i = 1]$,

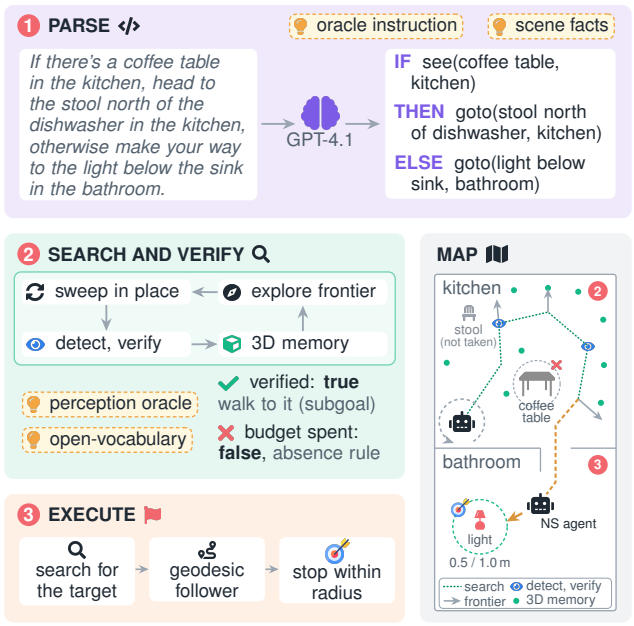


Figure 2. The neurosymbolic agent on one episode: the instruction is parsed into a program (1), each condition is resolved by the search-and-verify loop (2), and the chosen action is executed (3). On the map the agent (🤖) sweeps in place (◌), moves to frontiers (➡) along the teal search path, checks candidates (👁) and stores recognized objects (●) until the budget is spent; no coffee table is verified (✗), so the ELSE route (orange) leads to the lamp inside the success radius (◌), and the faded stool is the THEN action not taken. Chips (💡) mark what each ablation of Table 5 replaces.

where $\text{Success}_i = 1$ when the final position of the agent is within τ of the goal, and report $\text{CSR} = N^{-1} \sum_{i=1}^{N} \text{CSR}_i$. Episodes with subgoals thus require all ordered branch subgoals to be completed and the goal to be reached. For episodes without subgoals CSR_i equals SR_i. By construction, $\text{CSR} \leq \text{SR}$ holds for every agent. Section S5 reports both subsets with counts and splits episodes with subgoals by whether the branch was completed and whether the goal was reached.

4. Neurosymbolic Agent

CONDVLN is designed to distinguish failures in conditional reasoning from failures in perception, exploration, and navigation. We use a paired comparison: the same agent runs the same episode twice, once with the conditional instruction and once with an *oracle instruction* that already names the correct branch ("go to the light below the sink in the bathroom" for the episode of Figure 2, whose kitchen has no coffee table). The difference in success is that agent's *branch cost*. It is measurable only for an agent that can execute the oracle instruction: one that fails with either sentence shows no difference, which says nothing about its branch reasoning, and one that the words do not steer, such as a reader of a goal marker on a map, shows none either. We therefore provide an agent that reads the text and acts on the scene, and ablate it with further oracles that each remove one source of difficulty.

Neurosymbolic agent. Figure 2 shows the three stages. GPT-4.1 parses the instruction into an IF/THEN/ELSE program whose conditions are predicates $\text{see}(object, room)$, true when an instance of the object is found in the named room, and whose actions are $\text{goto}(target)$ commands. A search-and-verify loop resolves one condition at a time, sweeping in place, detecting on every frame, lifting detections through depth into a 3D memory, and exploring frontiers under a per-condition budget: a verified instance makes the condition true and the agent walks to it, which is the subgoal that BSA credits. An exhausted budget declares the condition false (the absence rule). Movement between waypoints uses the simulator's shortest-path follower with ground-truth pose, the usual local-planner assumption for modular agents; GPT-4.1 also confirms each candidate crop and labels room types in multi-room houses. Perception is a closed-set detector and a frame-level presence classifier fine-tuned on 15k to 18k views per environment rendered in a 60% training split of each environment's scenes (the evaluation sets span all scenes; restricting them to the held-out scenes changes no trend, Section S7). A second reference, the *trained policy*, is Qwen2.5-VL-3B fine-tuned with LoRA on 140k privileged-controller demonstration steps in the training scenes; it maps instruction and recent frames directly to actions and tests whether imitation closes the gap.

Ablations. Three oracles each remove one source of difficulty (💡 chips in Figure 2) while episodes, targets, and metrics stay fixed. The *oracle instruction* is the plain route of the correctly resolved branch. *Scene facts* are the conditional sentence plus the truth of each condition in words: the agent still parses and selects the branch but need not perceive the condition, so comparing it with the oracle instruction isolates the logic. The *perception oracle* replaces detector, verifier, and scans with the scene graph, which lists every vocabulary object in view within 7 m. A fourth variant, *open-vocabulary perception*, swaps the fine-tuned detector and classifier for off-the-shelf YOLO-World and a VLM grid scan; scene facts and the perception oracle are run on this variant, and the oracle instruction on both.

5. Experimental Results

Experiment overview. Our experiments test whether current VLN agents can handle the logical conditional structure introduced by CONDVLN. In particular, we ask four questions: (i) which agents perform best across environments, (ii) how performance changes as conditional instruction structure becomes more complex, (iii) whether failures

Table 3. Main results on CONDVLN, all categories pooled: SR, SPL, branch selection accuracy (BSA), and conditional success rate (CSR), all in %, on the verified sets (Section 5.1) under one scoring rule for every agent; VLN-Zero†: goal marker, distance readout, and map crop removed; shading: best (red), second (orange), third (yellow) per column.

Model	ReplicaCAD				AI2-THOR				Gibson				Matterport3D			
	SR	SPL	BSA	CSR	SR	SPL	BSA	CSR	SR	SPL	BSA	CSR	SR	SPL	BSA	CSR
NaVid	0.4	0.1	11.9	0.0	2.0	1.4	14.0	0.8	2.0	1.0	11.0	0.8	0.8	0.5	7.6	0.4
NaVILA	1.0	0.6	11.2	0.6	5.0	2.8	25.3	3.4	2.8	2.0	7.0	1.4	2.6	1.8	5.4	0.4
CA-Nav	0.6	0.5	12.0	0.4	2.2	1.5	14.1	1.2	1.0	0.5	6.1	0.6	1.0	0.4	4.7	0.8
Open-Nav	0.6	0.3	11.8	0.4	5.8	2.9	25.6	4.0	2.4	1.7	8.7	1.4	2.4	1.1	6.6	0.8
VLN-Zero†	0.4	0.4	9.7	0.2	1.6	1.1	16.1	0.8	0.8	0.8	4.7	0.2	0.4	0.4	3.0	0.2
StreamVLN	0.8	0.3	14.8	0.2	5.2	3.8	19.9	2.4	4.8	3.8	11.6	2.6	2.0	1.7	5.2	0.6
SNav-7B	1.2	1.1	16.2	0.8	5.8	2.2	18.1	2.4	3.8	3.0	10.8	2.2	1.4	1.3	7.4	0.6
Trained policy (ours)	5.8	2.3	31.4	3.8	13.6	4.4	57.6	9.6	4.4	0.9	26.3	2.8	2.2	1.0	17.5	1.0
Neurosymbolic agent (ours)	34.4	5.3	71.6	27.4	31.6	5.0	75.5	28.0	8.0	1.5	28.9	5.8	3.4	0.5	21.2	2.6

Table 4. Scaling analysis on CONDVLN; the base case (d=1, ℓ=1) is shared by both sweeps and reported once. Cells give BSA and CSR in % over 332 to 336 verified episodes; dagger and shading as in Table 3; nested-chain and per-environment cells in Section S4.

Model	Base		Logical Depth Sweep				Chain Length Sweep			
	d=1, ℓ=1		d=2, ℓ=1		d=3, ℓ=1		d=1, ℓ=2		d=1, ℓ=3	
	BSA	CSR	BSA	CSR	BSA	CSR	BSA	CSR	BSA	CSR
NaVid	9.04	0.30	9.04	0.90	12.13	0.90	10.71	0.00	15.06	0.00
NaVILA	11.14	2.11	8.89	2.11	11.43	0.60	14.88	1.49	15.06	1.20
CA-Nav	9.34	1.81	7.08	0.60	8.68	0.30	11.31	0.89	12.35	0.60
Open-Nav	12.95	3.01	8.58	0.60	11.48	0.60	15.77	2.08	18.98	2.71
VLN-Zero†	7.83	1.20	6.02	0.00	8.03	0.00	11.31	0.30	11.45	0.60
StreamVLN	9.34	1.51	9.64	1.20	15.27	1.20	13.99	1.79	17.77	1.51
SNav-7B	14.76	3.31	9.49	0.30	13.32	2.40	16.37	1.79	15.96	0.90
Trained policy (ours)	35.24	6.93	28.01	3.01	31.19	1.80	35.12	5.06	39.46	3.61
Neurosymbolic agent (ours)	46.99	19.58	48.34	15.96	47.11	15.57	53.57	15.77	50.90	13.86

are driven primarily by nested logical depth or by sequential chain length, and (iv) what an agent that checks the conditions gains and loses. The benchmark coverage and dataset statistics are summarized in Figure 1 and Table 2. Here, we focus on model evaluation using standard VLN metrics (SR and SPL) together with reasoning-specific metrics introduced in Section 3.2 (BSA and CSR).

5.1. Experimental Setup

Verified evaluation sets. All results use the verified evaluation sets of Section 3.1.5 (Table 2, Section S6).

Evaluated Agents. We evaluate seven VLN agents as baselines: NaVid, NaVILA, CA-Nav, Open-Nav, VLN-Zero [2] (with its goal marker, distance readout, and goal-centered map crop removed), and the public LLaVA-Video agents StreamVLN [22] and SNav-7B [24] (the NavSpace agent), together with the two agents of Section 4.

Implementation Details. All experiments are conducted using a unified evaluation interface built on VLN-CE-compatible infrastructure to ensure consistency across environments. Episodes are executed with a budget of 500 actions. Each baseline runs in its released evaluation code with our episodes and measures ported in; SNav-7B, which emits STOP at its first step on conditional sentences, has STOP suppressed for its first eight actions. The success-distance threshold is set to a tighter threshold than reported baselines: 0.5 m on ReplicaCAD (single-room scenes) and 1.0 m on the other three environments. The 3 m radius of R2R-CE would cover most of a single room, so success there would no longer tell whether the named object was reached (Section S8 rescores every agent at 0.5, 1, and 3 m). For conditional tasks, agents are not given explicit branch annotations beyond the natural-language instruction. They must implicitly infer the branch condition, evaluate the relevant scene evidence, and execute the corresponding navigation behavior.

5.2. Main Results

We present the main benchmark results across all four datasets and instruction-complexity categories in Table 3. General-purpose, non-custom VLN agents such as NaVid and NaVILA struggle across all datasets, exhibiting low and unstable CSR. This indicates that current training paradigms do not prepare agents for conditional instructions in 3D scenes.

StreamVLN and SNav-7B score in the same low range as NaVid and NaVILA. The zero-shot LLM-based agents fare no better at the benchmark radius. CA-Nav ignores the instruction until it detects the first landmark, which rarely happens in our dataset, so the sentence seldom steers its path. Pooled over environments, Open-Nav keeps one in seven of the successes it scores at its native 3 m radius, and VLN-Zero without its goal marker rarely reaches a goal. The neurosymbolic agent leads every column except SPL on the two real-scan environments, by five to thirty times the best baseline on the single-room scenes and narrowly on Matterport3D. As it earns its successes by searching, its successful negative-branch paths run 3 to 7× the shortest path, which lowers SPL (Section S5). Overall, these results show that standard VLN metrics alone are insufficient for evaluating conditional navigation.

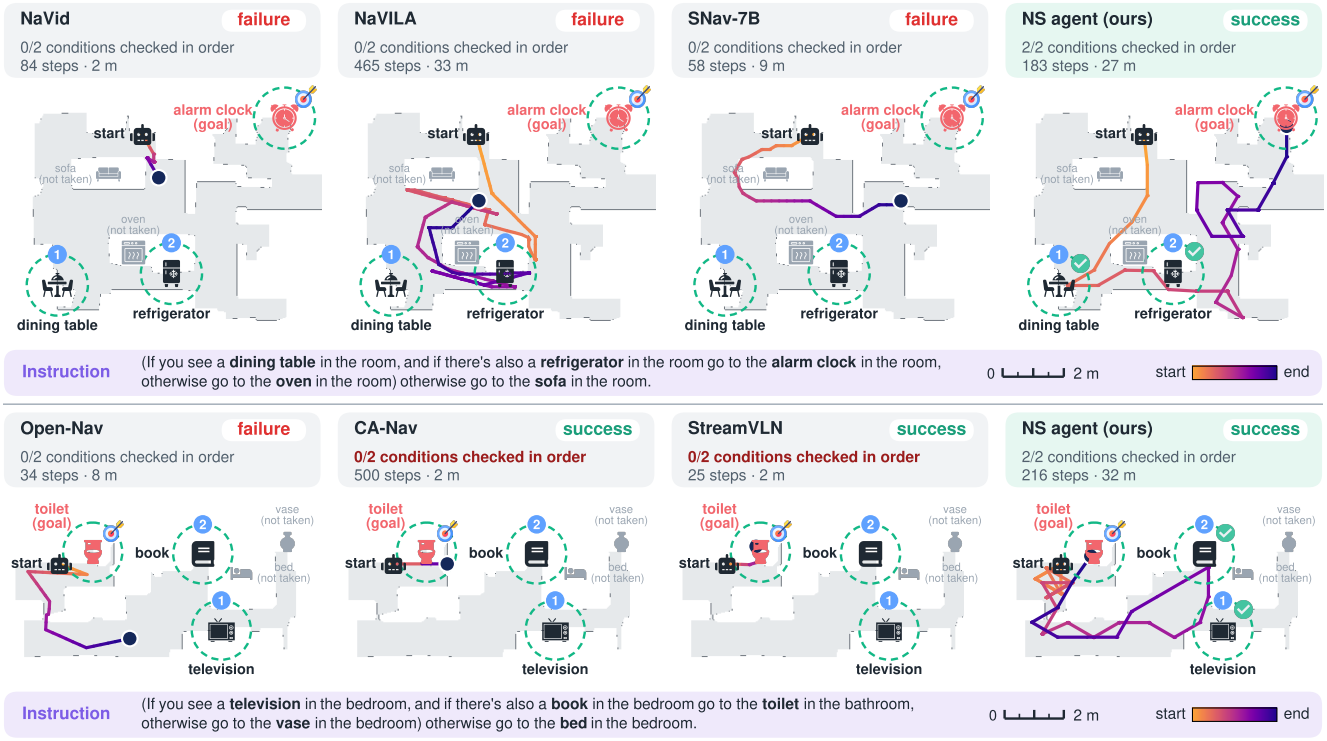


Figure 3. Paths of selected agents on two nested episodes (top: AI2-THOR; bottom: a Gibson house), one agent per panel, coloured from start to end; each row's instruction is in the box below it with the object names in bold. The start (), the condition objects numbered in check order (1, 2) and the goal object (coral,) sit inside their success radius (); marks a condition object an agent verified in place, faded icons are the destinations of the branches not taken, and a maroon condition count flags a success that checked nothing. Only the neurosymbolic agent completes the branch in both episodes. In the bottom row CA-Nav and StreamVLN are counted as successes only because the toilet happens to be within 1 m of where they stop: they never go near the television or the book, so they did not verify the conditions and reached the right goal without knowing it was right. Section S9 shows every agent on these and more example episodes.

Table 5. Ablations, reported as success rate (SR, %): each indented row hands the agent above it the named oracle (Section 4); open-vocabulary perception uses off-the-shelf models instead of the fine-tuned ones. Paired differences and McNemar tests in Section S3.

Variant	ReplicaCAD	AI2-THOR	Gibson	MP3D
Neurosymbolic agent	34.4	31.6	8.0	3.4
+ oracle instruction	37.4	44.4	12.2	3.4
Open-vocabulary perception	26.4	20.0	8.0	5.8
+ oracle instruction	29.6	35.6	13.6	7.6
+ scene facts	29.8	30.4	12.6	6.6
+ perception oracle	55.6	73.2	8.2	5.2
+ perception oracle + oracle instruction	57.8	72.2	13.6	9.6
Trained policy	5.8	13.6	4.4	2.2
+ oracle instruction	13.0	11.6	8.0	3.2

5.3. Depth and Chain Scaling

Table 4 further decomposes instruction complexity along two controlled axes: logical depth and chain length. This analysis isolates whether performance degradation is driven primarily by hierarchical conditional nesting or by long-horizon sequential dependencies.

The baselines remain weak in every configuration, with no trend in CSR over depth or chain length (Section S4): they rarely reach the named target with either instruction (Section S1), so depth and chain length have nothing to act on. The neurosymbolic agent holds its BSA within a seven-point band across all complexities: checking each condition explicitly seems to keep branch following stable as the logic deepens. Its CSR falls by 18 to 29% from the single-condition category (d=1, ℓ=1) to the deeper and longer ones. With the oracle instruction its success is flat across complexities (22 to 26%), as is its CSR on episodes where it judges every condition correctly (20 to 27%; Table S4). A single condition is judged correctly 71 to 85% of the time, and most errors declare a present object absent; because errors compound, the share of episodes with all conditions judged correctly drops from 82% (d=1, ℓ=1) to 58 to 67% (two or three conditions), and CSR with any misjudged condition stays at or below 5% (Section S4).

5.4. Neurosymbolic Agent and Ablations

Neurosymbolic agent versus baselines. Table 3 puts the neurosymbolic agent ahead of every baseline on SR, BSA, and CSR in every environment, by a wide margin on the single-room scenes. The lead comes from following the branch. Figure 3 shows the paths the agents actually took on two episodes, which makes their degree of conditional instruction following and verification visible: the neurosymbolic agent visits and verifies both condition objects before

heading to the goal, while the baselines leave them unvisited. The second episode also shows how a baseline can be counted a success without checking anything: its goal lies 1.4 m from the start, and CA-Nav and StreamVLN reach it by barely moving, which is the behaviour behind the gap between SR and CSR in Table 3. Section S9 shows further episodes with every agent. On episodes with a true condition, whose branch therefore carries subgoals, the agent completes the branch on 70% of the episodes it solves, and reaching the goal without completing the branch (a wrong branch or a shortcut) accounts for 6% of its failures. The baselines reach the goal on 1 to 3% of these episodes, and in only 1 to 7 episodes of 1,343 after completing the branch (Section S5). They also gain nothing from having the branch decided for them: the oracle instruction changes the success of the three we tested (NaVid, NaVILA, CA-Nav) by about two points at most (Section S1), so it can only speak for an agent that executes a two-destination instruction.

Ablations: where the cost goes. Ablations (Table 5; paired differences in Section S3) remove one source of difficulty at a time. The resolved branch raises success by 3.0, 12.8, 4.2, and 0.0 points on ReplicaCAD, AI2-THOR, Gibson, and Matterport3D (9%, 41%, 53%, and 0% of the conditional run's successes). With the condition truths given in words (scene facts), the agent matches the oracle instruction within 1 point except on AI2-THOR, where it trails by 5.2 points. The perception oracle, by contrast, multiplies the success of the open-vocabulary variant by 2.1 on ReplicaCAD and 3.7 on AI2-THOR, and on these single-room scenes the branch cost vanishes under perfect perception (4 to 5 points remain on the real-scan houses). Fine-tuned perception recovers 22 to 27% of that gap. The resolved branch multiplies the success of the trained policy by 1.8 to 2.2 on ReplicaCAD and Gibson, a larger relative cost than the neurosymbolic agent pays: imitation teaches partial branch following, not condition resolution. On the real-scan houses perfect perception within 7 m changes little: in 41% of episodes a present condition object never comes into view. The ablations thus order the difficulties. On single-room scenes the main cost is recognition, above all deciding that an object is absent; on real-scan houses it is exploration; branch logic costs least, at most 15.6 points and nothing on single-room scenes once perception is perfect (Sections S2 and S3). Progress on conditional VLN is therefore a matter of evidence acquisition, which fine-tuned perception addresses and imitation does not.

6. Conclusion and Limitations

We introduced CondVLN, a benchmark for evaluating conditional branching in vision-language navigation. By programmatically generating conditional navigation instructions from grounded scene graphs, CondVLN enables controlled evaluation of branch reasoning, condition grounding, and navigation execution. The benchmark spans four embodied navigation environments including AI2-THOR, MP3D, Gibson, and ReplicaCAD, providing diverse navigation settings and reasoning complexity. Our experiments show that current VLN agents perform poorly on conditional instructions, revealing failure modes not captured by standard navigation metrics alone. We further introduced a neurosymbolic agent that checks each condition explicitly: it follows the branch that every baseline bypasses, outperforms them on every metric but SPL, and its ablations attribute the remaining cost to evidence acquisition rather than to branch logic. Overall, CondVLN provides a reusable testbed for studying neurosymbolic reasoning and conditional decision-making in embodied navigation.

CondVLN currently focuses on VLN-CE-compatible indoor navigation datasets, limiting evaluation to existing simulator ecosystems. The benchmark also inherits imperfections from the underlying scene scans and semantic annotations across the four environments, which can affect perceptual grounding and condition verification. In addition, our spatial relation generation relies on fixed geometric heuristics that may not generalize across all scene distributions or environment types. Finally, the ablations of our neurosymbolic agent use scene-graph information that real agents do not have, and the agent itself relies on the simulator's shortest-path follower for low-level control. Future work will extend CondVLN to outdoor and open-world settings, and develop more adaptive relation-generation methods.

References

[1] Peter Anderson, Qi Wu, Damien Teney, Jake Bruce, Mark Johnson, Sabine Süsstrunk, Mathieu Salzmann, Anton Van Den Hengel, and Stephen Gould. Vision-and-language navigation: Interpreting visually-grounded navigation instructions in real environments. In *CVPR*, 2018. 1, 2

[2] Neel P. Bhatt, Yunhao Yang, Rohan Siva, Pranay Samineni, Tyler Ingebrand, Zhangyang Wang, and Ufuk Topcu. VLN-Zero: Rapid exploration and cache-enabled neurosymbolic vision-language planning for zero-shot transfer in robot navigation. *arXiv preprint arXiv:2509.18592*, 2025. 7

[3] Angel Chang, Angela Dai, Thomas Funkhouser, Maciej Halber, Matthias Niessner, Manolis Savva, Shuran Song, Andy Zeng, and Yinda Zhang. Matterport3d: Learning from rgb-d data in indoor environments. *International Conference on 3D Vision (3DV)*, 2017. 3

[4] Kehan Chen, Dong An, Yan Huang, Rongtao Xu, Yifei Su, Yonggen Ling, Ian Reid, and Liang Wang. Constraint-aware zero-shot vision-language navigation in continuous environments. *arXiv preprint arXiv:2412.10137*, 2024. 2

[5] An-Chieh Cheng, Yandong Ji, Zhaojing Yang, Xueyan Zou, Jan Kautz, Erdem Biyik, Hongxu Yin, Sifei Liu, and Xiao-

long Wang. Navila: Legged robot vision-language-action model for navigation. In *RSS*, 2025. 2

[6] Matt Deitke, Winson Han, Alvaro Herrasti, Aniruddha Kembhavi, Eric Kolve, Roozbeh Mottaghi, Jordi Salvador, Dustin Schwenk, Eli VanderBilt, Matthew Wallingford, Luca Weihs, Mark Yatskar, and Ali Farhadi. RoboTHOR: An Open Simulation-to-Real Embodied AI Platform. In *CVPR*, 2020. 3

[7] Matt Deitke, Eli VanderBilt, Alvaro Herrasti, Luca Weihs, Jordi Salvador, Kiana Ehsani, Winson Han, Eric Kolve, Ali Farhadi, Aniruddha Kembhavi, and Roozbeh Mottaghi. ProcTHOR: Large-Scale Embodied AI Using Procedural Generation. In *NeurIPS*, 2022. Outstanding Paper Award. 3

[8] Yifei Dong, Fengyi Wu, Qi He, Heng Li, Minghan Li, Zebang Cheng, Yuxuan Zhou, Jingdong Sun, Qi Dai, Zhi-Qi Cheng, and Alexander G Hauptmann. Ha-vln: A benchmark for human-aware navigation in discrete-continuous environments with dynamic multi-human interactions, real-world validation, and an open leaderboard, 2025. 2

[9] Yi Du, Taimeng Fu, Zhipeng Zhao, Shaoshu Su, Zitong Zhan, Qiwei Du, Zhuoqun Chen, Bowen Li, and Chen Wang. VL-Nav: Neuro-symbolic reasoning-based vision-language navigation. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2026. arXiv:2502.00931. 2

[10] Zebin Han, Xudong Wang, Baichen Liu, Qi Lyu, Zhenduo Shang, Jiahua Dong, Lianqing Liu, and Zhi Han. Seqwalker: Sequential-horizon vision-and-language navigation with hierarchical planning, 2026. 2

[11] Noriaki Hirose, Catherine Glossop, Dhruv Shah, and Sergey Levine. Omnivla: An omni-modal vision-language-action model for robot navigation, 2025. 2

[12] Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Daniel Gordon, Yuke Zhu, Abhinav Gupta, and Ali Farhadi. AI2-THOR: An Interactive 3D Environment for Visual AI. *arXiv*, 2017. 1, 2, 3

[13] Jacob Krantz, Erik Wijmans, Arjun Majumdar, Dhruv Batra, and Stefan Lee. Beyond the nav-graph: Vision-and-language navigation in continuous environments. In *European Conference on Computer Vision*, pages 104–120. Springer, 2020. 2

[14] Alexander Ku, Peter Anderson, Roma Patel, Eugene Ie, and Jason Baldridge. Room-across-room: Multilingual vision-and-language navigation with dense spatiotemporal grounding. In *EMNLP*, 2020. 1, 2

[15] Sihao Lin, Zerui Li, Xunyi Zhao, Gengze Zhou, Liuyi Wang, Rong Wei, Rui Tang, Juncheng Li, Hanqing Wang, Jiangmiao Pang, Anton van den Hengel, Jiajun Liu, and Qi Wu. Vlnverse: A benchmark for vision-language navigation with versatile, embodied, realistic simulation and evaluation, 2025. 2

[16] Yanyuan Qiao, Wenqi Lyu, Hui Wang, Zixu Wang, Zerui Li, Yuan Zhang, Mingkui Tan, and Qi Wu. Open-nav: Exploring zero-shot vision-and-language navigation in continuous environment with open-source llms. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2025. 2

[17] Xinshuai Song, Weixing Chen, Yang Liu, Weikai Chen, Guanbin Li, and Liang Lin. Towards long-horizon vision-language navigation: Platform, benchmark and method. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2025. 2

[18] Hung-Ting Su, Ting-Jun Wang, Jia-Fong Yeh, Min Sun, and Winston H. Hsu. Vln-nf: Feasibility-aware vision-and-language navigation with false-premise instructions, 2026. ACL 2026. 2

[19] Andrew Szot, Alex Clegg, Eric Undersander, Erik Wijmans, Yili Zhao, John Turner, Noah Maestre, Mustafa Mukadam, Devendra Chaplot, Oleksandr Maksymets, Aaron Gokaslan, Vladimir Vondrus, Sameer Dharur, Franziska Meier, Wojciech Galuba, Angel Chang, Zsolt Kira, Vladlen Koltun, Jitendra Malik, Manolis Savva, and Dhruv Batra. Habitat 2.0: Training home assistants to rearrange their habitat. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2021. 1, 2, 3

[20] Francesco Taioli, Stefano Rosa, Alberto Castellini, Lorenzo Natale, Alessio Del Bue, Alessandro Farinelli, Marco Cristani, and Yiming Wang. Mind the Error! Detection and Localization of Instruction Errors in Vision-and-Language Navigation. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 12993–13000, 2024. 2

[21] Tianhang Wang, Xinhai Li, Fan Lu, Tianshi Gong, Jiankun Dong, Weiyi Xue, Sanqing Qu, Chenjia Bai, and Guang Chen. Conavbench: Collaborative long-horizon vision-language navigation benchmark. In *The Fourteenth International Conference on Learning Representations*, 2026. 2

[22] Meng Wei, Chenyang Wan, Xiqian Yu, Tai Wang, Yuqiang Yang, Xiaohan Mao, Chenming Zhu, Wenzhe Cai, Hanqing Wang, Yilun Chen, Xihui Liu, and Jiangmiao Pang. Streamvln: Streaming vision-and-language navigation via slowfast context modeling. *arXiv preprint arXiv:2507.05240*, 2025. 7

[23] Fei Xia, Amir R. Zamir, Zhi-Yang He, Alexander Sax, Jitendra Malik, and Silvio Savarese. Gibson env: real-world perception for embodied agents. In *Computer Vision and Pattern Recognition (CVPR), 2018 IEEE Conference on*. IEEE, 2018. 1, 2, 3

[24] Haolin Yang, Yuxing Long, Zhuoyuan Yu, Zihan Yang, Minghan Wang, Jiapeng Xu, Yihan Wang, Ziyan Yu, Wenzhe Cai, Lei Kang, and Hao Dong. Navspace: How navigation agents follow spatial intelligence instructions, 2025. 2, 7

[25] Jiazhao Zhang, Kunyu Wang, Rongtao Xu, Gengze Zhou, Yicong Hong, Xiaomeng Fang, Qi Wu, Zhizheng Zhang, and He Wang. Navid: Video-based vlm plans the next step for vision-and-language navigation. *Robotics: Science and Systems*, 2024. 2

[26] Gengze Zhou, Yicong Hong, Zun Wang, Xin Eric Wang, and Qi Wu. Navgpt-2: Unleashing navigational reasoning capability for large vision-language models. *arXiv preprint arXiv:2407.12366*, 2024. 2